NOUS AI ACADEMY / INTENSIVE FILE

TECHNICAL LEARNING FILE

DEEP-07

Calculus and Optimization

Derivatives and stable learning

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended

FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply calculus and optimization with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

UNIT 01 / CONCEPT

Limits and Change: Concept

01 OBJECTIVE

Explain and apply limits and change using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Limits and Change is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Limits, Change are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should limits and change be used, and what observable evidence would make that choice defensible? Build a small local example that applies limits and change, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Limits and Change in one precise sentence and name one assumption.
2. Create a two-step example of limits and change and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKFLOW

Limits and Change: Workflow

01 OBJECTIVE

Explain and apply limits and change using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Limits and Change is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to limits and change.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies limits and change, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Limits and Change in one precise sentence and name one assumption.
2. Create a two-step example of limits and change and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKED EXAMPLE

Limits and Change: Worked Example

01 OBJECTIVE

Explain and apply limits and change using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies limits and change, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns limits and change into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Limits and Change in one precise sentence and name one assumption.
2. Create a two-step example of limits and change and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / FAILURE ANALYSIS

Limits and Change: Failure Analysis

01 OBJECTIVE

Explain and apply limits and change using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how limits and change can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to limits and change. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Limits and Change in one precise sentence and name one assumption.
2. Create a two-step example of limits and change and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / APPLIED LAB

Limits and Change: Applied Lab

01 OBJECTIVE

Explain and apply limits and change using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of limits and change into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies limits and change, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Limits and Change in one precise sentence and name one assumption.
2. Create a two-step example of limits and change and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / CONCEPT

Partial Derivatives: Concept

01 OBJECTIVE

Explain and apply partial derivatives using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Partial Derivatives is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Partial, Derivatives are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should partial derivatives be used, and what observable evidence would make that choice defensible? Build a small local example that applies partial derivatives, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Partial Derivatives in one precise sentence and name one assumption.
2. Create a two-step example of partial derivatives and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKFLOW

Partial Derivatives: Workflow

01 OBJECTIVE

Explain and apply partial derivatives using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Partial Derivatives is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to partial derivatives.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies partial derivatives, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Partial Derivatives in one precise sentence and name one assumption.
2. Create a two-step example of partial derivatives and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKED EXAMPLE

Partial Derivatives: Worked Example

01 OBJECTIVE

Explain and apply partial derivatives using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies partial derivatives, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns partial derivatives into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Partial Derivatives in one precise sentence and name one assumption.
2. Create a two-step example of partial derivatives and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / FAILURE ANALYSIS

Partial Derivatives: Failure Analysis

01 OBJECTIVE

Explain and apply partial derivatives using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how partial derivatives can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to partial derivatives. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Partial Derivatives in one precise sentence and name one assumption.
2. Create a two-step example of partial derivatives and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / APPLIED LAB

Partial Derivatives: Applied Lab

01 OBJECTIVE

Explain and apply partial derivatives using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of partial derivatives into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies partial derivatives, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Partial Derivatives in one precise sentence and name one assumption.
2. Create a two-step example of partial derivatives and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / CONCEPT

Chain Rule: Concept

01 OBJECTIVE

Explain and apply chain rule using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Chain Rule is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Chain, Rule are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should chain rule be used, and what observable evidence would make that choice defensible? Build a small local example that applies chain rule, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Chain Rule in one precise sentence and name one assumption.
2. Create a two-step example of chain rule and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKFLOW

Chain Rule: Workflow

01 OBJECTIVE

Explain and apply chain rule using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Chain Rule is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to chain rule.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies chain rule, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Chain Rule in one precise sentence and name one assumption.
2. Create a two-step example of chain rule and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKED EXAMPLE

Chain Rule: Worked Example

01 OBJECTIVE

Explain and apply chain rule using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies chain rule, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns chain rule into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Chain Rule in one precise sentence and name one assumption.
2. Create a two-step example of chain rule and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / FAILURE ANALYSIS

Chain Rule: Failure Analysis

01 OBJECTIVE

Explain and apply chain rule using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how chain rule can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to chain rule. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Chain Rule in one precise sentence and name one assumption.
2. Create a two-step example of chain rule and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / APPLIED LAB

Chain Rule: Applied Lab

01 OBJECTIVE

Explain and apply chain rule using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of chain rule into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies chain rule, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Chain Rule in one precise sentence and name one assumption.
2. Create a two-step example of chain rule and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / CONCEPT

Jacobians: Concept

01 OBJECTIVE

Explain and apply jacobians using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Jacobians is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Jacobians are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should jacobians be used, and what observable evidence would make that choice defensible? Build a small local example that applies jacobians, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Jacobians in one precise sentence and name one assumption.
2. Create a two-step example of jacobians and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKFLOW

Jacobians: Workflow

01 OBJECTIVE

Explain and apply jacobians using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Jacobians is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to jacobians.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies jacobians, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Jacobians in one precise sentence and name one assumption.
2. Create a two-step example of jacobians and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKED EXAMPLE

Jacobians: Worked Example

01 OBJECTIVE

Explain and apply jacobians using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies jacobians, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns jacobians into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Jacobians in one precise sentence and name one assumption.
2. Create a two-step example of jacobians and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / FAILURE ANALYSIS

Jacobians: Failure Analysis

01 OBJECTIVE

Explain and apply jacobians using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how jacobians can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to jacobians. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Jacobians in one precise sentence and name one assumption.
2. Create a two-step example of jacobians and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / APPLIED LAB

Jacobians: Applied Lab

01 OBJECTIVE

Explain and apply jacobians using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of jacobians into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies jacobians, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Jacobians in one precise sentence and name one assumption.
2. Create a two-step example of jacobians and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / CONCEPT

Gradient Descent: Concept

01 OBJECTIVE

Explain and apply gradient descent using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Gradient Descent is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Gradient, Descent are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should gradient descent be used, and what observable evidence would make that choice defensible? Build a small local example that applies gradient descent, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Gradient Descent in one precise sentence and name one assumption.
2. Create a two-step example of gradient descent and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKFLOW

Gradient Descent: Workflow

01 OBJECTIVE

Explain and apply gradient descent using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Gradient Descent is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to gradient descent.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies gradient descent, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Gradient Descent in one precise sentence and name one assumption.
2. Create a two-step example of gradient descent and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKED EXAMPLE

Gradient Descent: Worked Example

01 OBJECTIVE

Explain and apply gradient descent using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies gradient descent, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns gradient descent into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Gradient Descent in one precise sentence and name one assumption.
2. Create a two-step example of gradient descent and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / FAILURE ANALYSIS

Gradient Descent: Failure Analysis

01 OBJECTIVE

Explain and apply gradient descent using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how gradient descent can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to gradient descent. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Gradient Descent in one precise sentence and name one assumption.
2. Create a two-step example of gradient descent and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / APPLIED LAB

Gradient Descent: Applied Lab

01 OBJECTIVE

Explain and apply gradient descent using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of gradient descent into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies gradient descent, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Gradient Descent in one precise sentence and name one assumption.
2. Create a two-step example of gradient descent and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / CONCEPT

Momentum: Concept

01 OBJECTIVE

Explain and apply momentum using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Momentum is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Momentum are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should momentum be used, and what observable evidence would make that choice defensible? Build a small local example that applies momentum, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Momentum in one precise sentence and name one assumption.
2. Create a two-step example of momentum and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKFLOW

Momentum: Workflow

01 OBJECTIVE

Explain and apply momentum using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Momentum is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to momentum.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies momentum, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Momentum in one precise sentence and name one assumption.
2. Create a two-step example of momentum and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKED EXAMPLE

Momentum: Worked Example

01 OBJECTIVE

Explain and apply momentum using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies momentum, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns momentum into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Momentum in one precise sentence and name one assumption.
2. Create a two-step example of momentum and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / FAILURE ANALYSIS

Momentum: Failure Analysis

01 OBJECTIVE

Explain and apply momentum using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how momentum can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to momentum. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Momentum in one precise sentence and name one assumption.
2. Create a two-step example of momentum and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / APPLIED LAB

Momentum: Applied Lab

01 OBJECTIVE

Explain and apply momentum using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of momentum into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies momentum, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Momentum in one precise sentence and name one assumption.
2. Create a two-step example of momentum and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / CONCEPT

Adaptive Optimizers: Concept

01 OBJECTIVE

Explain and apply adaptive optimizers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Adaptive Optimizers is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Adaptive, Optimizers are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should adaptive optimizers be used, and what observable evidence would make that choice defensible? Build a small local example that applies adaptive optimizers, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Adaptive Optimizers in one precise sentence and name one assumption.
2. Create a two-step example of adaptive optimizers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKFLOW

Adaptive Optimizers: Workflow

01 OBJECTIVE

Explain and apply adaptive optimizers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Adaptive Optimizers is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to adaptive optimizers.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies adaptive optimizers, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Adaptive Optimizers in one precise sentence and name one assumption.
2. Create a two-step example of adaptive optimizers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKED EXAMPLE

Adaptive Optimizers: Worked Example

01 OBJECTIVE

Explain and apply adaptive optimizers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies adaptive optimizers, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns adaptive optimizers into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Adaptive Optimizers in one precise sentence and name one assumption.
2. Create a two-step example of adaptive optimizers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / FAILURE ANALYSIS

Adaptive Optimizers: Failure Analysis

01 OBJECTIVE

Explain and apply adaptive optimizers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how adaptive optimizers can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to adaptive optimizers. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Adaptive Optimizers in one precise sentence and name one assumption.
2. Create a two-step example of adaptive optimizers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / APPLIED LAB

Adaptive Optimizers: Applied Lab

01 OBJECTIVE

Explain and apply adaptive optimizers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of adaptive optimizers into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies adaptive optimizers, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Adaptive Optimizers in one precise sentence and name one assumption.
2. Create a two-step example of adaptive optimizers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / CONCEPT

Convexity: Concept

01 OBJECTIVE

Explain and apply convexity using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Convexity is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Convexity are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should convexity be used, and what observable evidence would make that choice defensible? Build a small local example that applies convexity, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Convexity in one precise sentence and name one assumption.
2. Create a two-step example of convexity and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKFLOW

Convexity: Workflow

01 OBJECTIVE

Explain and apply convexity using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Convexity is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to convexity.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies convexity, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Convexity in one precise sentence and name one assumption.
2. Create a two-step example of convexity and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKED EXAMPLE

Convexity: Worked Example

01 OBJECTIVE

Explain and apply convexity using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies convexity, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns convexity into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Convexity in one precise sentence and name one assumption.
2. Create a two-step example of convexity and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / FAILURE ANALYSIS

Convexity: Failure Analysis

01 OBJECTIVE

Explain and apply convexity using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how convexity can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to convexity. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Convexity in one precise sentence and name one assumption.
2. Create a two-step example of convexity and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / APPLIED LAB

Convexity: Applied Lab

01 OBJECTIVE

Explain and apply convexity using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of convexity into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies convexity, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Convexity in one precise sentence and name one assumption.
2. Create a two-step example of convexity and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / CONCEPT

Optimization Diagnostics: Concept

01 OBJECTIVE

Explain and apply optimization diagnostics using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Optimization Diagnostics is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Calculus and Optimization, the terms Optimization, Diagnostics are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should optimization diagnostics be used, and what observable evidence would make that choice defensible? Build a small local example that applies optimization diagnostics, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Optimization Diagnostics in one precise sentence and name one assumption.
2. Create a two-step example of optimization diagnostics and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKFLOW

Optimization Diagnostics: Workflow

01 OBJECTIVE

Explain and apply optimization diagnostics using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Optimization Diagnostics is a central part of calculus and optimization because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to optimization diagnostics.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies optimization diagnostics, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Optimization Diagnostics in one precise sentence and name one assumption.
2. Create a two-step example of optimization diagnostics and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKED EXAMPLE

Optimization Diagnostics: Worked Example

01 OBJECTIVE

Explain and apply optimization diagnostics using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies optimization diagnostics, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns optimization diagnostics into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Optimization Diagnostics in one precise sentence and name one assumption.
2. Create a two-step example of optimization diagnostics and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / FAILURE ANALYSIS

Optimization Diagnostics: Failure Analysis

01 OBJECTIVE

Explain and apply optimization diagnostics using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how optimization diagnostics can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to optimization diagnostics. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Optimization Diagnostics in one precise sentence and name one assumption.
2. Create a two-step example of optimization diagnostics and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / APPLIED LAB

Optimization Diagnostics: Applied Lab

01 OBJECTIVE

Explain and apply optimization diagnostics using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of optimization diagnostics into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies optimization diagnostics, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
Model: y_hat = w * x + b
Loss:  L = (y_hat - y)^2
Gradient: dL/dw = 2 * (y_hat - y) * x
Update: w_next = w - learning_rate * dL/dw
```

05 PRACTICE

1. Define Optimization Diagnostics in one precise sentence and name one assumption.
2. Create a two-step example of optimization diagnostics and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating calculus and optimization. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.